A Comparison of Symbolic and Neuro-Symbolic Strategies for LTLf Runtime Monitoring

Author name

Date

Abstract

TODO

1 Introduction

Linear Temporal Logic (LTL) [13] is a logic formalism for expressing temporal properties of reactive systems. It extends traditional propositional logic with modal operators, allowing the specification of rules that must hold *through time*, and it is interpreted over traces of discrete events. Variants of LTL interpreted over finite traces have been proposed for modeling finite, but length-unbounded, process executions (see Sec. 2.1), making it suitable for finite-horizon problems that often arise in the context of AI and BPM. Temporal logics have provided one of the most popular tool in Formal Methods for verification of hardware or software system behaviors. In particular, model checking is the problem of verifying whether a model of a system meets a given specification. Despite the importance and the progress made in model checking, it is not always possible to check the model against the specification. This is due to the inherent complexity of the system or the fact that the model is only partially correct or not known. In such cases, one can resort to simpler verification approaches, such as runtime monitoring, where the system's execution is continuously observed on-the-fly and any permanent violation or fulfillment of the specified property is immediately detected.

Standard symbolic techniques for both model checking and runtime monitoring have achieved great success, but they also suffer for some inherent limitation. Among the others there is the difficulty in dealing with the increasing growth of big data and the lack of flexibility in dealing with highly unpredictable environments, say e.g. evniroments with noisy sensors or probabilistic uncertainties, and adapting the specification when soft violations could happen. Conversely, pure deep learning architectures, such as Recurrent Neural Networks (RNNs) and Transformers, excel at modeling high-dimensional, noisy data distributions, but they likely generate outputs that violate critical safety constraints, as they lack a mechanism to adhere to formal logics. This naturally leads to the use of hybrid approaches, which can possibly overcome some of the mentioned limitations.

In this paper, we will compare different techniques for runtime monitoring, from purely symbolic methods to the state-of-the-art neuro-symbolic architectures. We start revisiting the neural-symbolic approach to runtime monitoring, originally proposed by [11, 12] through the RuleRunner system [10]. The authors claimed that the resulting neural monitors could exploit GPU parallelism for efficient verification, outperforming purely symbolic mechanisms. Since then, some significant developments have reshaped the landscape. First, LTL_f [2] has become the standard formalism for

temporal reasoning over finite traces in AI, bringing with it mature tools for compiling formulas into DFAs. Then, differentiable automata representations such as DeepDFA [14] have emerged, enabling the integration of temporal logic knowledge into neural architectures through gradient-based learning. Possibly include other NeSy approaches. We present a systematic comparison of three paradigms for LTL_f runtime monitoring: (i) purely symbolic DFA-based monitoring, (ii) rule-based neural-symbolic monitoring in the RuleRunner tradition, and (iii) automata-based neural-symbolic monitoring using DeepDFA.
TODO: Mention briefly the experimental results obtained.

2 Background

2.1 An Historical Overview of Temporal Logics over Finite Traces

Studying LTL over finite traces gained significant attention [5, 8]. The original RuleRunner system [10] refers to FLTL as a finite-trace variant of LTL. As already mentioned, in the last decade LTL_f has established as the standard language for modeling finite traces of executions. Notably, FLTL and LTL_f share essentially the same syntax and semantics, offering two next operators, with a strong next operator X which is false at the last position if no successor exists, and a weak next operator W which is true at the last position (see Sec. 2.2 for details). FLTL makes explicit use of an *END* symbol for signaling the last element of the trace, but this can be easily expressed with the weak next operator by $END \equiv Wfalse$. In this paper, we use LTL_f as the specification language for all three monitoring paradigms we compare.
TODO: Mention other finite-trace variants.

2.2 Syntax and Semantics

Given a finite set $\mathcal{P}$ of atomic propositions, an LTL_f formula φ has the form:

$$\varphi ::= \top \mid \bot \mid a \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid X\varphi \mid \varphi_1 U \varphi_2,$$

where $a \in \mathcal{P}$. We use $\top$ and $\bot$ to denote $True$ and $False$, respectively. X (Strong Next) and U (Until) are temporal operators. Other temporal operators can be derived as follows: W (Weak Next) is defined as $W\varphi \equiv \neg X \neg\varphi$, R (Release) is defined as $\varphi_1 R \varphi_2 \equiv \neg(\neg\varphi_1 U \neg\varphi_2)$, G (globally) is defined as $G\varphi \equiv \bot R \varphi$, and F (eventually) is defined as $F\varphi \equiv \top U \varphi$.

A trace $\boldsymbol{\sigma} = (\sigma_1, \sigma_2, \ldots, \sigma_T)$ is a sequence of propositional assignments to the propositions in $\mathcal{P}$, where $\sigma_t \in 2^{\mathcal{P}}$ is the set of all and only propositions that are true at instant t. Additionally, $|\boldsymbol{\sigma}| = T$ denotes the length of the trace. Note that, if the propositional symbols in $\mathcal{P}$ are all *mutually exclusive*, i.e., the domain produces exactly one symbol true at each step, then we have $\sigma_t \in \mathcal{P}$. Let $last = |\boldsymbol{\sigma}| - 1$. We inductively define when an LTL_f formula φ is true at an instant i (for $0 \leq i \leq last$), written $\boldsymbol{\sigma}, i \models \varphi$, as follows:

- $\boldsymbol{\sigma}, i \models a$ iff $a \in \boldsymbol{\sigma}_i$, for $a \in \mathcal{P}$;
- $\boldsymbol{\sigma}, i \models \neg\varphi$ iff $\boldsymbol{\sigma}, i \not\models \varphi$;
- $\boldsymbol{\sigma}, i \models \varphi_1 \wedge \varphi_2$ iff $\boldsymbol{\sigma}, i \models \varphi_1$ and $\boldsymbol{\sigma}, i \models \varphi_2$;
- $\boldsymbol{\sigma}, i \models X\varphi$ iff $i < last$ and $\boldsymbol{\sigma}, i+1 \models \varphi$;

- $\boldsymbol{\sigma}, i \models \varphi_1 \mathcal{U} \varphi_2$ iff for some j such that $i \leq j \leq last$, we have $\boldsymbol{\sigma}, j \models \varphi_2$, and for all k such that $i \leq k < j$, we have $\boldsymbol{\sigma}, k \models \varphi_1$.

We say that $\boldsymbol{\sigma}$ satisfies ϕ, written $\boldsymbol{\sigma} \models \varphi$, if $\boldsymbol{\sigma}, 0 \models \varphi$. A formula φ is satisfiable if it is true wrt some trace $\boldsymbol{\sigma}$, and is valid, if it is true wrt every trace $\boldsymbol{\sigma}$.

In the following, we will use $sub(\varphi)$ to denote the set of proper subformulas of φ, and we will refer to it as the syntactic closure of φ. Given a formula φ and a propositional assignment σ_t at time t, we are also interested in computing the residual formula, i.e the formula that has to be satisfied at time $t+1$. As explained in details in Sec. **??**, a residual formula can be computed using formula progression [], which returns a positive Boolean formula on $sub(\varphi) \cup sub(F\top) \cup sub(G\bot)$. We will use $cl(\varphi)$ to denote the set of residual formulas, and we will refer to it as the residual closure of φ .

2.3 Symbolic Monitoring

Deterministic Finite Automata. Any LTL$_f$ formula φ can be translated into a Deterministic Finite Automaton (DFA) [2] $A_\varphi = (2^{\mathcal{P}}, Q, q_0, \delta, F)$, where $2^{\mathcal{P}}$ is the automaton alphabet, Q is the finite set of states, $q_0 \in Q$ is the initial state, $\delta : Q \times 2^{\mathcal{P}} \to Q$ is the transition function, and $F \subseteq Q$ is the set of final states. We also define the extended transition function over traces $\delta^* : Q \times (2^{\mathcal{P}})^* \to Q$ recursively as:

$$\begin{aligned} &\delta^*(q, \epsilon) = q \\ &\delta^*(q, \sigma + \boldsymbol{x}) = \delta^*(\delta(q, \sigma), \boldsymbol{x}), \end{aligned} \tag{1}$$

where $\sigma \in 2^{\mathcal{P}}$ is a symbol and $\boldsymbol{x} \in (2^{\mathcal{P}})^*$ is a trace. The automaton accepts the trace $\boldsymbol{\sigma}$ iff $\delta^*(q_0, \boldsymbol{\sigma}) \in F$, and in that case we say that $\boldsymbol{\sigma}$ belongs to the language of the automaton, denoted as $\mathcal{L}(A_\varphi)$. We have that φ and A_φ are equivalent because, for any trace $\boldsymbol{\sigma} \in (2^{\mathcal{P}})^*$:

$$\boldsymbol{\sigma} \in \mathcal{L}(A_\varphi) \iff \boldsymbol{\sigma} \vDash \varphi. \tag{2}$$

We can rely on the use of DFAs for several reasoning tasks, like model checking, planning and synthesis. In particular, monitoring formula φ becomes a deterministic walk over the automaton A_φ: the monitor keeps a single current state q, initialized to q_0, and on observing the time point $\sigma_t \in 2^{\mathcal{P}}$ it advances $q \leftarrow \delta(q, \sigma_t)$. Because the automaton is fixed and deterministic, each step costs a single transition lookup, independent of the trace seen so far and of the size of φ. In the following, DFA-based monitoring will serve as the exact baseline against which the neuro-symbolic paradigms will be measured.

Verdicts and early termination. Online monitoring requires a three-valued runtime semantics [1]: at each step the monitor emits SATISFY, VIOLATE, or UNDECIDED, the last meaning that the observed prefix can still be extended into both an accepting and a non-accepting continuation. A definite verdict can be issued as soon as the outcome is settled regardless of the future, which corresponds to two structural properties of the current state, both precomputed once by backward reachability over A_φ. A state is a *trap* if no accepting state is reachable from it: reaching a trap means φ is permanently violated, so the monitor halts with VIOLATE. Dually, a state is an *accepting sink* if every state reachable from it (including itself) is accepting: the monitor halts with SATISFY. With trap and sink sets materialized at compile time, the per-step verdict reduces to two set-membership tests on top of the transition lookup. As long as the current state is neither, the

verdict is UNDECIDED and monitoring continues. Some specifications never reach a trap or a sink, so STEP returns UNDECIDED for the entire trace. An example is the response pattern $G(a \rightarrow Fb)$, which can always be satisfied by a later b and always be rescued from violation, hence has neither an accepting sink nor a trap. For these the verdict is only binary at the end of the trace, and the monitor reports a final verdict by testing acceptance of the state reached, i.e. SATISFY iff $\delta^*(q_0, \boldsymbol{\sigma}) \in F$ and VIOLATE otherwise.

3 Rule-based Neuro-Symbolic Monitoring: RuleRunner

3.1 Architecture

A RuleRunner system is a tuple $\langle S, R_E, R_R \rangle$, where S is the state of the system, R_E is the set of *evaluation rules*, and R_R is the set of *reactivation rules*. In details:

- The sate S contains observations, active rules, and truth evaluations. $R[\phi]$ denotes an active rule, while $[\phi]V$ denotes a 3-values truth evaluation, with $V \in \{T, F, ?\}$.
- Evaluation rules are used to compute the truth value of a formula ϕ under verification, given the truth values of its direct subformulas ψ_i. They have the form $R[\phi], [\psi_1]V, ..., [\psi_n]V \rightarrow [\phi]V$. For instance, $R[a \vee b], [a]T \rightarrow [a \vee b]T$, because the disjuction is evaluated to *true* if one of the two disjuncts is true.
- Reactivation rules are used when a formula ϕ is evaluated to undecided, i.e. $[\phi]?$, in which case that formula and its subformulas are scheduled to be monitored again at the next time point of the trace. They have the form $[\phi]? \rightarrow R[\phi], R[\psi_1], ..., R[\psi_n]$. For instance, $[\Diamond b]? \rightarrow R[b], R[\Diamond b]$, because if $\Diamond b$ is undecided, we need to re-evaluate b and $\Diamond b$ at the next step.

Both active rules and truth evaluations can be enriched by a *qualifier*, such as B (binary), L (left) and R (right), which provides additional information recording which operands within a formula/subformula are still being watched. For instance, $R[a \wedge \Diamond b], [a]T \rightarrow [a \wedge \Diamond b]?R$, because if the left conjunct a is true, it is still needed to check the truth value right conjunct $\Diamond b$ to assign a truth value to the conjunction.

Verification of a trace against a formula proceeds iteratively. At the beginning of each iteration (corresponding to the monitoring of a time point), the state S contains a set of rule names corresponding to the subformulas to be checked at that time point. First, the procedure includes observations of the current time point in the state S and fires the evaluation rules repeatedly incrementing the state itself until it reaches a fixpoint. In practice, this means computing the truth values of all subformulas $sub(\phi)$ of ϕ in a bottom-up way, from simple atoms to ϕ itself (see Figure ...). If the root evaluates to true (resp. false), the monitor halts with verdict SATISFY (resp. VIOLATE). Otherwise, the reactivation rules are fired, and the result constitute the state of the next iteration. Note that in this case the state is not expanded, but replaced by the result of reactivation rules. This is used to flush all the previous truth evaluation, which are to be computed from scratch at the new time point. The procedure terminates with a decided verdict because of special END-triggered evaluation rules, that returns either true or false if we consumed the whole trace. A RuleRunner system can be translated in a logic program, which in turn can be encoded in a Recurrent Neural Network with a single hidden layer, s.t. each evaluation and reactivation rule

is a mapping from the input layer to the output one. For a detailed description of RuleRunner and verification algorithm see [11, 10].

The rule system underlies two behaviorally-equivalent neural encodings, both of which we evaluate in Section 6. In the *flat* encoding [11], the whole rule set is compiled into a single recurrent network following the standard CILP translation []: one hidden unit per rule, weights realizing each Horn clause, and the fixpoint of the evaluation phase recovered by iterating the forward pass until the fixpoint is met. In the *structured* encoding [12], the same rules are instead laid out as one CILP subnetwork per parse-tree node, wired according to the syntax of φ. This modular form is the substrate for *local learning*: because parameters are attached to syntactically-local subnetworks, adaptation can be confined to the subtree responsible for a misclassification rather than diffused across a monolithic network.

3.2 The Nested-Temporal Limitation

RuleRunner's defining design choice is to maintain *a single shared state, with exactly one truth register per syntactic subformula.* This is what gives it a constant, compile-time rule count, a fixed-size network, and a non-branching state, and it is well suited to many flat temporal specifications. The problem is that the architecture *assumes each subformula has exactly one live truth value per time point.* This assumption is violated precisely when a temporal operator is nested inside the operand of another operator that may reinstall that operand across successive time points: the outer operator spawns a fresh instance of the operand while a prior instance of the *same* subformula is still resolving, and the two distinct live obligations are conflated onto the one shared register.

Consider the two formulas $\varphi_g = F(a \wedge b)$ and $\varphi_b = F(a \wedge Xb)$, which differ only by a single next operator, monitored over the same trace $\boldsymbol{\sigma} = (\{a\}, \emptyset, \{b\})$. Neither formula is satisfied: φ_g would require a time point where a and b hold *together* (but a occurs only at $t = 0$ and b only at $t = 2$), and φ_b would require a time point t with $a \in \sigma_t$ and $b \in \sigma_{t+1}$ (but the only a is at $t = 0$,and is not followed by a b at $t = 1$). The correct verdict in both cases is therefore VIOLATE, but RuleRunner can provide the correct answer only for the first formula.

First, consider φ_g. The run is:

t	σ_t	evaluation	reactivation
0	$\{a\}$	$[a]T, [b]F \to [a \wedge b]F$ $[a \wedge b]F \to [F(a \wedge b)]?$	$\to R[F(a \wedge b)], R[a], R[b], R[a \wedge b]$
1	$\emptyset$	$[a]F, [b]F \to [a \wedge b]F$ $[a \wedge b]F \to [F(a \wedge b)]?$	$\to R[F(a \wedge b)], R[a], R[b], R[a \wedge b]$
2	$\{b\}$	$[a]F, [b]T \to [a \wedge b]F$ $[a \wedge b]F \to [F(a \wedge b)]?$ $[F(a \wedge b)]? \to_{\text{END}} [F(a \wedge b)]F$	$\to$ VIOLATE

Now, consider φ_b, In this case, the run diverges:

t	σ_t	evaluation	reactivation
0	$\{a\}$	$R[Xb] \to [Xb]?$ $[a]T, [Xb]? \to [a \wedge Xb]?R$ $[a \wedge Xb]?R \to [F(a \wedge Xb)]?$	$\to R[Xb]M, R[b]$ $\to R[a \wedge Xb]R$ $\to R[F(a \wedge Xb)], R[a], R[Xb], R[a \wedge Xb]$
1	$\emptyset$	$R[Xb] \to [Xb]?$ $R[Xb]M, [b]F \to [Xb]F$ $R[a \wedge Xb]R, [Xb]? \to [a \wedge Xb]?R$ $R[a \wedge Xb]R, [Xb]T \to [a \wedge Xb]T$ $R[a \wedge Xb], [a]F \to [a \wedge Xb]F$ $[a \wedge Xb]?R \to [F(a \wedge Xb)]?$	$\to R[Xb]M, R[b]$ $\to R[a \wedge Xb]R$ $\to R[F(a \wedge Xb)], R[a], R[Xb], R[a \wedge Xb]$
2	$\{b\}$	$R[Xb] \to [Xb]?$ $R[Xb]M, [b]T \to [Xb]T$ $R[a \wedge Xb]R, [Xb]T \to [a \wedge Xb]T$ $R[a \wedge Xb], [a]F \to [a \wedge Xb]F$ $[a \wedge Xb]T \to [F(a \wedge Xb)]T$	$\to$ SATISFY (wrong)

The divergence stems from RuleRunner's single shared state, which keeps exactly one truth register per syntactic subformula. The decisive step is the F reactivation. At every time point, an F that remains undecided reinstalls a fresh $R[Xb]$ that will defer again, while the previous time point's deferral is still resolving via $R[Xb]M$. As mentioned previously, the state is a single flat set of truth evaluations and rule names with no position index. A reinstalling operator can place, into that one set, a freshly-deferred instance of a subformula while a prior instance of the same subformula is still resolving. The two instances are indistinguishable, and the next evaluation pass conflates them, possibly holding contradictory values for one subformula at once. In other words, the reactivation phase does not preserve the single-instance-per-subformula invariant across the whole formula.

This is a structural consequence of the one-literal-per-subformula encoding, and does not depend on the encoding of RuleRunner or on the implementation artifacts. The conflation cannot be repaired at the output layer. Consider φ_b again, the two traces $\boldsymbol{\sigma}_A = (\{a\}, \emptyset, \{b\})$ (ground truth VIOLATE) and $\boldsymbol{\sigma}_B = (\emptyset, \{a\}, \{b\})$ (ground truth SATISFY) for φ_b. After the second time point both present the *identical* conflicted register $[Xb] = \{F, ?\}$, yet the correct resolutions are opposite: $\boldsymbol{\sigma}_A$ needs the F to win, $\boldsymbol{\sigma}_B$ needs the ? to win. Any monitor whose decision is a function of the shared register alone must answer identically on both, so no priority order over the register can be correct on both traces.

3.3 Progression-Based Reactivation

The counterexample of Section 3.2 should not be read as a failure of the RuleRunner idea as a whole. It isolates a more precise problem: the original construction carries information across time using registers addressed only by the syntactic subformulas of the input formula. The local evaluation rules are not the source of the error. Given the values of the direct subformulas at one time point, the extended truth tables compute the intended three-valued result. What fails is the reactivation phase: when the same subformula is reinstalled from different temporal contexts, the distinct residual meanings of that subformula are written into the same register.

The repair we consider keeps the two ingredients that make RuleRunner distinctive: a rule-based evaluation/reactivation cycle, and the possibility of translating the resulting rule system into a recurrent CILP-style network. The change is in what is carried across time. Instead of carrying

only truth registers for subformulas of the original input formula, the repaired monitor carries residual formulas obtained by progression [**?**, **?**]. The monitor still uses formula registers, and these registers are still associated with syntactic formulas; however, the relevant formulas are now taken from the residual closure of the input formula, not only from its original parse tree.

RuleRunner, fixed. Formula progression gives a principled account of what remains to be checked after reading one observation. Given φ and $s \in 2^{\mathcal{P}}$, let $\mathsf{prog}(\varphi, s)$ denote the formula that has to hold on the remaining suffix after s has been consumed, such that for every finite continuation $\boldsymbol{\sigma}$, the following holds:

$$s \cdot \boldsymbol{\sigma} \models \varphi \quad \Longleftrightarrow \quad \boldsymbol{\sigma} \models \mathsf{prog}(\varphi, s),$$

Formally, formula progression is defined recursively as follows:

$$\begin{aligned}
\mathsf{prog}(a, s) &= \top \text{ if } a \in s, \text{ and } \bot \text{ otherwise,} \\
\mathsf{prog}(\varphi_1 \wedge \varphi_2, s) &= \mathsf{prog}(\varphi_1, s) \wedge \mathsf{prog}(\varphi_2, s), \\
\mathsf{prog}(\varphi_1 \vee \varphi_2, s) &= \mathsf{prog}(\varphi_1, s) \vee \mathsf{prog}(\varphi_2, s), \\
\mathsf{prog}(X\varphi, s) &= \varphi \wedge F\top, \\
\mathsf{prog}(\varphi_1 U \varphi_2, s) &= \mathsf{prog}(\varphi_2, s) \vee \big(\mathsf{prog}(\varphi_1, s) \wedge (\varphi_1 U \varphi_2)\big).
\end{aligned}$$

We assume that every time *prog* is computed, it is followed by a Boolean simplification. Derived operators are handled through their definitions.

The repaired monitor carries a single *residual formula* $\rho_t \in cl(\varphi)$, the formula that the remaining suffix must satisfy. It is initialized to $\rho_0 = \varphi$ and updated by progression

$$\rho_{t+1} = \mathsf{nf}\big(\mathsf{prog}(\rho_t, \sigma_t)\big)$$

where nf is a semantics-preserving Boolean normalization discussed below. The verdict is read off the residual itself: $\rho_t \equiv \top$ means that every continuation satisfies φ, and the monitor halts with SATISFY; $\rho_t \equiv \bot$ means that none does, and it halts with VIOLATE; otherwise the verdict is UNDECIDED. As in the original system, evaluating a formula at a time point is a bottom-up computation over its parse tree. In details, the RuleRunner state S_t at time t holds the observations of σ_t, the active rules $R[\psi]$, and the truth evaluations $[\psi]V$ for $\psi \in sub(\rho_t)$. These are exactly the registers the original system used, only instantiated over the residual rather than over φ; and, exactly as before, they are *flushed* at each time point rather than carried. What is carried across time is the residual alone. Note that since only the residual is progressed, the undecided subformulas $[\psi]?$ of ρ_t are not reactivated as before, but flushed as every other truth evaluation.

The residual is carried not as a monolithic formula but in *factored* form, as the set of its top-level conjuncts:

$$\Sigma_t = \mathsf{split}_\wedge(\rho_t)$$

where $\mathsf{split}_\wedge(\chi_1 \wedge \chi_2) = \mathsf{split}_\wedge(\chi_1) \cup \mathsf{split}_\wedge(\chi_2)$, $\mathsf{split}_\wedge(\top) = \emptyset$, and any formula that is not a top-level conjunction is kept as a single element. Progression is a homomorphism for conjunction, $\mathsf{prog}(\chi_1 \wedge \chi_2, s) = \mathsf{prog}(\chi_1, s) \wedge \mathsf{prog}(\chi_2, s)$, so the elements of Σ_t may be progressed independently and re-collected, leaving the update above unchanged. The factorization is used to reduce the size of the system. If we carry the whole residual, the monitor would need one register per reachable residual, i.e. it would have the size of $cl(\varphi)$. Carrying the conjuncts instead makes it a *multi-hot* vector over the top-level conjuncts, and the gap can be exponential. For $\varphi = Fa_1 \wedge ... \wedge Fa_n$ the

reachable residuals is given by all subsets of still-pending conjuncts, so $|cl(\varphi)| = 2^n$, whereas the top-level conjuncts are the n formulas Fa_i, giving a linear bound on the number of registers needed.

To summarize, the RuleRunner two-phase schema is repaired as followed:

1. **Evaluation.** The observation σ_t is injected into S_t and the evaluation rules R_E are fired bottom-up until a fixpoint is reached, computing $[\psi]V$ for every $\psi \in sub(\rho_t)$.
2. **Reactivation.** The residual is factorized and each top-level conjunct is progressed independently. The results are then re-collected into the new residual, i.e.,

$$\rho_{t+1} = \bigwedge_{\chi \in \Sigma_t} [\mathsf{nf}(\mathsf{prog}(\chi, \sigma_t))]$$

The cost of the repair. One may wonder whether the new RuleRunner construction comes with a cost. Actually, it depends on how the monitor is realized.

In the *eager* realization, the residual closure $cl(\varphi)$ and its syntactic closure $sub(cl(\varphi)) = \bigcup_{\chi \in cl(\varphi)} sub(\chi)$ are computed once, and all corresponding evaluation and progression rules are compiled into a fixed finite rule system. Using the CILP translation, we can then map it to a recurrent network exactly as in the original RuleRunner approach. The price is paid on two axes. The first is the closure itself, since the register set is bounded by $cl(\varphi)$, not φ as before. The second axis comes from the progression schema, since $\mathsf{prog}(\chi, \sigma_t)$ depends on the observation. Compiling it into a fixed set of clauses therefore requires expanding over the observation space. In the worst case, the size of the monitor is $|cl(\varphi)| \cdot 2^{|\mathcal{P}|}$. This is similar to what happens in the dense DeepDFA tensor (see Sec. 4).

In the *lazy* realization, only the closure $sub(\rho_t)$ of the current residual is instantiated, one time point at a time. Neither axis is ever enumerated: the set of registers is bounded by the parse tree of ρ_t, that is $sub(\rho_t)$, and the progression is computed only for that particular residual against the current observation, not expanded over $2^{|\mathcal{P}|}$. The cost here is that we cannot encode the monitor as a recurrent network, since the rule set is no longer fixed, but dynamically generated. Conversely, we can use a *sequence of distinct feedforward neural networks*, once per time step. This means that cross-trace batching is lost, since traces in a batch hold different residuals and share no weight matrix.

Soundness and Completeness. Now, we can prove that the repaired construction of RuleRunner is sound and complete.

Theorem 1. *Let φ be an* LTL_f *formula and let $RR_\varphi^{\mathsf{prog}}$ be the (eager) progression-based RuleRunner monitor constructed as above. Then, for every finite non-empty trace $\boldsymbol{\sigma}$, the monitor returns* SATISFY *iff $\boldsymbol{\sigma} \models \varphi$, and it returns* VIOLATE *iff $\boldsymbol{\sigma} \not\models \varphi$. Moreover, any verdict returned before the last cell is sound for all finite continuations of the prefix read so far.*

Proof. Let $\boldsymbol{\sigma} = (\sigma_0, \ldots, \sigma_{T-1})$ be a finite non-empty trace. For $0 \le t < T$, let Σ_t be the active root set before reading cell σ_t, that is, after the prefix $\sigma_0 \cdots \sigma_{t-1}$ has been consumed. Let

$$\rho_t = \bigwedge_{\chi \in \Sigma_t} \chi$$

be the residual it denotes. By construction,

$$\Sigma_0 = \mathsf{split}_\wedge(\mathsf{nf}(\varphi)),$$

and therefore ρ_0 is equivalent to φ.

We first use the defining property of progression. For every formula χ, every non-final cell s, and every non-empty finite continuation $\boldsymbol{\tau}$,

$$s \cdot \boldsymbol{\tau} \models \chi \quad \Longleftrightarrow \quad \boldsymbol{\tau} \models \mathsf{prog}(\chi, s). \tag{3}$$

This property follows by structural induction on χ. The atomic and Boolean cases are immediate from the definition of prog. For $X\chi$, since $\boldsymbol{\tau}$ is non-empty, $s\cdot\boldsymbol{\tau} \models X\chi$ iff $\boldsymbol{\tau} \models \chi$, which is exactly the clause $\mathsf{prog}(X\chi, s) = \chi$. The weak-next case is identical on non-final cells. For until, $s\cdot\boldsymbol{\tau} \models \chi_1 U \chi_2$ iff either χ_2 holds at the current cell, or χ_1 holds at the current cell and $\chi_1 U \chi_2$ holds on the suffix; by the induction hypothesis this is equivalent to

$$\boldsymbol{\tau} \models \mathsf{prog}(\chi_2, s) \vee \big(\mathsf{prog}(\chi_1, s) \wedge (\chi_1 U \chi_2)\big).$$

The derived operators inherit the property from their definitions.

We now prove the residual invariant:

$$\sigma_0 \cdots \sigma_{t-1} \cdot \boldsymbol{\tau} \models \varphi \quad \Longleftrightarrow \quad \boldsymbol{\tau} \models \rho_t \tag{4}$$

for every $0 \le t < T$ and every non-empty finite continuation $\boldsymbol{\tau}$.

For $t = 0$, the claim holds because ρ_0 is equivalent to φ. Assume it holds for some $t < T - 1$. By the reactivation rule,

$$\rho_{t+1} \equiv \mathsf{nf}\left(\bigwedge_{\chi \in \Sigma_t} \mathsf{prog}(\chi, \sigma_t)\right).$$

Since $\rho_t = \bigwedge_{\chi\in\Sigma_t} \chi$ and progression distributes over conjunction up to logical equivalence, this is equivalent to

$$\rho_{t+1} \equiv \mathsf{nf}\big(\mathsf{prog}(\rho_t, \sigma_t)\big).$$

Therefore, for every non-empty continuation $\boldsymbol{\tau}$,

$$\begin{aligned}
\sigma_0 \cdots \sigma_t \cdot \boldsymbol{\tau} \models \varphi \quad &\Longleftrightarrow \quad \sigma_t \cdot \boldsymbol{\tau} \models \rho_t \quad \text{by the induction hypothesis,} \\
&\Longleftrightarrow \quad \boldsymbol{\tau} \models \mathsf{prog}(\rho_t, \sigma_t) \quad \text{by Equation (3),} \\
&\Longleftrightarrow \quad \boldsymbol{\tau} \models \rho_{t+1} \quad \text{because } \mathsf{nf} \text{ preserves semantics.}
\end{aligned}$$

Thus the invariant holds at $t + 1$.

It remains to consider the last cell. Applying the invariant with $t = T - 1$ and continuation (σ_{T-1}) gives

$$\boldsymbol{\sigma} \models \varphi \quad \Longleftrightarrow \quad (\sigma_{T-1}) \models \rho_{T-1}.$$

By definition of last, the right-hand side is equivalent to

$$\mathsf{last}(\rho_{T-1}, \sigma_{T-1}) = \top.$$

The final evaluation rules compute precisely this value. Hence the monitor returns SATISFY exactly on traces satisfying φ, and VIOLATE otherwise.

Finally, the operational register construction realizes the symbolic transition used in the proof. At time t, the state $S_t = \mathit{sub}(\rho_t)$ contains every subformula needed to evaluate and progress the active roots. Since the eager monitor allocates registers for all formulas in $\mathit{sub}(\mathit{cl}(\varphi))$, every such

support formula has a corresponding register, and the bottom-up computation can be carried out by finitely many rules. Only the active roots in Σ_t are reactivated, so support subformulas do not acquire an independent temporal meaning. Therefore the rule-based monitor implements exactly the residual update used in the invariant.

If an early verdict is returned, the current residual is respectively valid or unsatisfiable over all finite continuations. By Equation (4), every completion of the prefix read so far then has the same truth value for φ, which proves soundness of early termination. □

The role of normalization. As described above, we use a Boolean normalization on the residual formulas produced by progression. It is important to notice that this is not strictly required for soundness, meaning that nf could be an the identity function and the monitor would still be sound. What normalization buys is *finiteness*. Progression's raw output may grow syntactically without bound. For instance, since $\mathsf{prog}(F\psi, s) = \mathsf{prog}(\psi, s) \vee F\psi$, a trace on which ψ keeps failing produces $\bot \vee F\psi$, then $\bot \vee (\bot \vee F\psi)$, and so on. Without normalization, every time point would allocate fresh registers and the eager compilation may not terminate. In other words, the residual formulas are finite *up to logical equivalence*, and nf is useful to make $cl(\varphi)$ a finite set of registers.

Furthermore, the normalization step can also help in keeping the residual formula compact and easy to evalutate. For compactness, note that $\mathsf{split}_\wedge$ only sees the conjuncts that nf exposes, so a normalization rewriting $(a \wedge b) \vee (a \wedge c)$ into $a \wedge (b \vee c)$ yields two conjuncts instead of just one. For evaluation, the normalization can be used to simplify the residual formulas, reducing the number of evaluation rules that need to be fired and detecting early termination.

The counterexample revisited. Consider again the formula $\varphi_b = F(a \wedge Xb)$ and the two traces $\boldsymbol{\sigma}_A = (\{a\}, \emptyset, \{b\})$ and $\boldsymbol{\sigma}_B = (\emptyset, \{a\}, \{b\})$. The residual formulas before each time step are:

$$\begin{array}{lllll} & \rho_0 & \xrightarrow{\sigma_1} \rho_1 & \xrightarrow{\sigma_2} \rho_2 & \xrightarrow{\sigma_3} \rho_3 \\ \boldsymbol{\sigma}_A: & F(a \wedge Xb) & \xrightarrow{\{a\}} b \vee F(a \wedge Xb) & \xrightarrow{\emptyset} F(a \wedge Xb) & \xrightarrow{\{b\}} false \\ \boldsymbol{\sigma}_B: & F(a \wedge Xb) & \xrightarrow{\emptyset} F(a \wedge Xb) & \xrightarrow{\{a\}} b \vee F(a \wedge Xb) & \xrightarrow{\{b\}} true \end{array}$$

For trace $\boldsymbol{\sigma}_A$, the residual $b \vee F(a \wedge Xb)$ appears after the first time step. At the second time step, b is false, so the disjunct disappears and the residual becomes $F(a \wedge Xb)$ again. At the last time step, the trace contains b, but the remaining formula $F(a \wedge Xb)$ is false on this last step, since $a \wedge Xb$ is false there. Hence the verdict is VIOLATE, as expected. For trace $\boldsymbol{\sigma}_B$, the residual $b \vee F(a \wedge Xb)$ appears one step later, immediately before the time step containing b. At the last time step, this residual is evaluated to true because the disjunct b holds. Hence the verdict is SATISFY.

This is exactly the information lost by the original one-register construction. The repaired monitor does not try to store two incompatible truth evaluations in the single register $[Xb]$. Instead, it stores the residual formula $b \vee F(a \wedge Xb)$, and it occurs at different time points in the two traces.

4 Automata-based Neuro-Symbolic Monitoring: DeepDFA

4.1 Architecture

DeepDFA [14] keeps the automaton of the symbolic paradigm but replaces its explicit state walk with a differentiable, tensorized one; we adopt the same forward formulation used for predictive

monitoring of business processes by [9]. In its original formulation DeepDFA is a *probabilistic relaxation* of a DFA: a probabilistic finite automaton whose transition tensor is real-valued and *learnable*, with each transition row constrained to the probability simplex (e.g. through a softmax or a Gumbel–Softmax relaxation of discrete choices [4, 6]), and the crisp automaton recovered only in the zero-temperature, one-hot limit. Accordingly, starting from the minimal DFA $A_\varphi = (2^{\mathcal{P}}, Q, q_0, \delta, F)$ of Section 2.2, the general object is a three-way tensor $T \in [0,1]^{|Q|\times|2^{\mathcal{P}}|\times|Q|}$ that is row-stochastic in its last index, $\sum_{q'} T[q,\sigma,q'] = 1$, together with an acceptance vector $v_o \in [0,1]^{|Q|}$. When the automaton is instead *compiled* from a known LTL_f formula — the setting of this paper's monitoring experiments — these parameters are fixed to their exact one-hot values, $T \in \{0,1\}^{|Q|\times|2^{\mathcal{P}}|\times|Q|}$ with $T[q,\sigma,q'] = 1$ iff $\delta(q,\sigma) = q'$, and $v_o \in \{0,1\}^{|Q|}$ with $v_o[q] = 1$ iff $q \in F$; this crisp instantiation is what the monitor uses, while the real-valued parameterization is the substrate for the gradient-based adaptation of Section 5. In either case the monitor state is no longer a single automaton state but a row vector $\mathbf{q}_t \in [0,1]^{|Q|}$ over states, initialized to the one-hot vector $\mathbf{q}_0 = v_i$ of q_0. Writing $T_\sigma = T[\,\cdot\,,\sigma,\cdot\,] \in [0,1]^{|Q|\times|Q|}$ for the (row-stochastic) transition matrix of symbol σ, a step is a single vector–matrix product

$$\mathbf{q}_t = \mathbf{q}_{t-1}\, T_{\sigma_t}, \qquad \mathrm{acc}_t = \mathbf{q}_t \cdot v_o, \tag{5}$$

where $\mathrm{acc}_t \in [0,1]$ is the acceptance score of the prefix read so far. On the compiled tensor every T_{σ_t} is a 0/1 deterministic-transition matrix and, on crisp observations, $\mathbf{q}_t$ stays one-hot, so the update reproduces the symbolic walk exactly: $\mathbf{q}_t$ is the indicator of $\delta^*(q_0, \sigma_1 \cdots \sigma_t)$. The three-valued verdict and early termination are then read off exactly as in Section 2.3, by testing membership of $\mathrm{argmax}_q\, \mathbf{q}_t[q]$ in the precomputed trap and accepting-sink sets, and the final verdict tests its acceptance.

4.2 Soft Observations and Differentiability

The reason to pay for this tensorization is that Equation (5) extends verbatim to *probabilistic* observations, which the symbolic walk cannot represent. Suppose a time point is delivered not as a crisp assignment but as a vector of per-atom probabilities $\mathbf{p}_t \in [0,1]^{\mathcal{P}}$, as produced by a neural perceptor or a noisy sensor. We replace the one-hot symbol selection by an *expected* transition matrix $M(\mathbf{p}_t) \in [0,1]^{|Q|\times|Q|}$, whose entry (q,q') is the probability that the guard of edge $q \to q'$ is satisfied under $\mathbf{p}_t$. Here our setting departs from the source formulation: the original DeepDFA and the process-monitoring work that adopts it assume *mutual exclusivity* — exactly one activity is true per step — so the alphabet is the set of atoms and the expected transition is a simple convex combination $M(\mathbf{p}_t) = \sum_\sigma p_\sigma\, T_\sigma$ of the one-hot per-symbol matrices, weighted by the categorical symbol probabilities. Because our propositional benchmark family admits conjunctions of simultaneously-true atoms (Section 2.2), we drop this assumption and instead compute each edge's guard-satisfaction probability compositionally over the boolean structure of the guard under an atom-independence assumption ($P(a) = p_a$, $P(\neg\psi) = 1 - P(\psi)$, $P(\psi_1 \wedge \psi_2) = P(\psi_1)P(\psi_2)$, $P(\psi_1 \vee \psi_2) = 1 - (1 - P(\psi_1))(1 - P(\psi_2))$). The update and acceptance score are unchanged,

$$\mathbf{q}_t = \mathbf{q}_{t-1}\, M(\mathbf{p}_t), \qquad \mathrm{acc}_t = \mathbf{q}_t \cdot v_o, \tag{6}$$

so that $\mathbf{q}_t$ propagates the perceptual uncertainty all the way to the verdict rather than discarding it through a hard 0.5 threshold, and acc_t is a soft acceptance score that the symbolic walk fundamentally cannot emit. Because each $M(\mathbf{p}_t)$ is a smooth function of $\mathbf{p}_t$ (and, if desired, of the

learnable parameters of T), the whole monitor is differentiable: gradients of a verdict loss flow back through the automaton, which is what makes DeepDFA the vehicle for gradient-based specification adaptation and, ultimately, end-to-end training of a perceptor. On crisp inputs $M(\mathbf{p}_t)$ collapses to T_{σ_t} and the probabilistic semantics is exact for any guard. On fractional inputs, however, the guarantee is conditional: the compositional score is exact — and $M(\mathbf{p}_t)$ genuinely row-stochastic, so acc_t is a well-defined acceptance *probability* — precisely when every guard is *read-once* (each atom occurring once), which holds for the LTL_f families we study after MONA's factoring. On a non-read-once guard the independence assumption double-counts shared atoms across disjuncts: the per-state out-guard probabilities no longer sum to one, $M(\mathbf{p}_t)$ ceases to be row-stochastic, and the raw acc_t is no longer a valid probability (it can exceed 1). It can be renormalized by the total propagated mass $\mathbf{q}_t \cdot \mathbf{1}$ to restore the range $[0, 1]$ — a no-op on read-once guards, where the mass is already one — but this restores range validity, not calibration; we return to this point, and to which probabilistic quantity a monitor *should* report, in Section 5 and the experiments.

4.3 The Alphabet Blow-up

DeepDFA's structural cost is dual to those of the other two paradigms. The transition tensor is indexed by the full alphabet $2^{\mathcal{P}}$, so a dense materialization is exponential in the number of atoms, $|T| = |Q|^2 \cdot 2^{|\mathcal{P}|}$. One can avoid building the dense tensor by evaluating the per-edge guard probabilities on the fly — the factored form underlying Equations (5)–(6), which never enumerates $2^{\mathcal{P}}$ — and this is what we use for specifications over many atoms. But the blow-up is not merely an implementation choice: when a single guard depends on all $|\mathcal{P}|$ atoms there is in general no sub-exponential set of symbols to sum over, and only the read-once circuit structure of a guard (which a flat DFA does not expose) permits its satisfaction probability to be computed in time linear in $|\mathcal{P}|$.

Concretely, the monitor admits two representations of the transition function, and we evaluate both. The *dense* representation materializes the full tensor T and executes each step as a single (batched) matrix product indexing the observed symbol; it is the fastest option whenever the alphabet fits in memory and is the natural showcase for cross-trace batching, but it becomes infeasible past roughly a dozen atoms, where $2^{|\mathcal{P}|}$ exhausts memory. The *factored* representation, developed next, never builds T and is what we use for specifications over many atoms.

4.4 The Factored Transition Representation

The factored representation exploits the fact that the DFA edges produced by MONA are labelled not by individual symbols but by *symbolic boolean guards* over the atoms (e.g. $a \wedge (b \vee c)$), each standing for the set of symbols that satisfy it. This is the same compact edge labelling that lets the symbolic walk sidestep the alphabet (Section 2.3); the factored mode carries it into the tensorized setting while preserving both exact crisp monitoring and a differentiable soft path. The construction is our own assembly of standard ingredients from Boolean-function representation and knowledge compilation — Shannon expansion, disjoint sum-of-products, and circuit-based probability evaluation [**?**] — rather than a component inherited from the DeepDFA literature; the idea of forming an *expected* transition operator from symbol/guard probabilities is shared with recent neurosymbolic-automata work [7, **?**], which likewise targets probabilistic inputs to automata in the non-mutually-exclusive setting.

We keep two complementary views of each edge guard, corresponding to the crisp and the soft regimes.

Crisp path: disjoint cube cover. At construction time each guard is expanded, by Shannon expansion, into a set of *disjoint* (mutually exclusive) cubes — partial assignments in which some atoms are required true, some required false, and the rest are don't-cares — stored as a pair of require-true/require-false integer masks over the atoms. Given a per-atom value vector $\mathbf{p}$, each cube evaluates to $\prod_a \left[1 - \mathrm{rt}_a(1 - p_a) - \mathrm{rf}_a\, p_a\right]$ (which is p_a for a required-true atom, $1 - p_a$ for a required-false atom, and 1 for a don't-care), and the cubes of all edges are summed into the $|Q| \times |Q|$ transition matrix by a single vectorized reduction, at a cost proportional to the number of cubes rather than to $2^{|\mathcal{P}|}$. On crisp 0/1 inputs each cube is 0/1 and, being mutually exclusive, the cubes of a state's out-guards partition the assignment space and sum to a 0/1 row-stochastic transition, so this path reproduces the symbolic walk exactly. This is the path our monitoring experiments time, and it is what keeps the factored curve flat in $|\mathcal{P}|$.

Crucially, the factored representation is *not* an unconditional escape from the alphabet blow-up: Shannon expansion of a guard on k atoms can produce up to $\Theta(2^k)$ disjoint cubes, so a guard that genuinely depends on all $|\mathcal{P}|$ atoms with no compact orthogonal cover reinstates the exponential cost in the number of cubes. What the factored form buys is a shift of the blow-up from "always $2^{|\mathcal{P}|}$" (the dense tensor) to "$2^{|\mathcal{P}|}$ only for adversarial guards": a genuine and often large win on the structured, read-once guards that arise in practice (our LTL_f families among them), and no asymptotic improvement on pathological ones. This is consistent with the finding of Section 4.3 — only the read-once circuit structure of a guard admits a sub-exponential evaluation — now made precise at the level of the representation.

Soft path: recursive guard probabilities. Separately, each guard is compiled into a differentiable closure that evaluates its satisfaction probability recursively over the boolean tree, using the independence rules of Equation (6); summing these per-edge closures yields the expected transition matrix $M(\mathbf{p}_t)$, differentiable in $\mathbf{p}_t$ and in the automaton parameters. This is the substrate for the soft observations and gradient-based adaptation of Section 4.2. On read-once guards the two views coincide even on fractional inputs, since the orthogonal-cube sum equals the recursive read-once probability; on crisp inputs both are exact for any guard. They part ways only on non-read-once guards with fractional inputs, and instructively so: the disjoint-cube sum computes the *exact* independent-atom marginal (it sums probabilities of mutually exclusive events), whereas the recursive closure double-counts shared atoms and can overshoot. We deliberately drive the soft readout from the recursive closure rather than the exact-marginal cube sum — both because the cube count can itself blow up, as noted above, and because retaining the overshoot keeps the non-read-once non-stochasticity of Section 4.2 observable rather than silently corrected. Which of these quantities is the "correct" probabilistic verdict for a monitor is itself a question we take up in Section 5.

Our scalability experiments plot the dense and factored paths side by side — the dense curve walling out at the $2^{|\mathcal{P}|}$ memory bound while the factored curve stays flat on the read-once families — which separates the alphabet blow-up as a *memory* wall from the residual per-step *compute* cost.

5 Theoretical Comparison

Having presented the three paradigms individually, we now compare them along the dimensions that matter for a monitor: what each can represent, where each breaks down, and — for the two neuro-symbolic paradigms — what a gradient means when the specification is adapted from data.

5.1 Three Achilles Heels

Each paradigm is exact and efficient on a large class of specifications, and each has a single structural weakness that is dual to the others', so that no one paradigm dominates.

- **Symbolic DFA — state-space blow-up.** The LTL_f-to-DFA translation is doubly exponential in the worst case, so the compiled automaton (and hence the compile-time cost and memory) can explode even when the runtime visits only a handful of states.
- **RuleRunner — nested-temporal conflation.** The one-register-per-subformula state cannot represent disjunctions of concurrently-live obligations, so it returns wrong verdicts on temporal-under-temporal nesting (Section 3.2); restoring completeness forces the state up to DFA size (Section 3.3).
- **DeepDFA — alphabet blow-up.** The transition tensor is indexed by $2^{\mathcal{P}}$, and guards depending on all atoms admit no sub-exponential symbol set to sum over unless their read-once circuit structure is exposed (Section 4.3).

Two clarifications keep this comparison honest. First, the state-space blow-up is not exclusive to the symbolic paradigm: DeepDFA is compiled from the *same* minimal DFA, so it inherits $|Q|$ (its tensor is $|Q|^2 \cdot 2^{|\mathcal{P}|}$) and suffers the same doubly-exponential worst case. We list state blow-up under the symbolic monitor because it is that monitor's *only* structural weakness, whereas for DeepDFA the *additional*, representation-specific cost that the symbolic monitor escapes is the alphabet axis. Second, the two neuro-symbolic-versus-symbolic heels are duals in a precise sense: a DFA transition function is a table with $|Q| \cdot 2^{|\mathcal{P}|}$ entries, and each paradigm compresses a different axis of it to obtain its advantage, paying on the axis it leaves explicit. The symbolic monitor compresses the $2^{\mathcal{P}}$ axis with symbolic guards — keeping an explicit current state, so its cost driver is $|Q|$ — and this is exactly why it cannot suffer an alphabet blow-up: it never enumerates symbols. DeepDFA (dense) compresses the state axis into a uniform, differentiable, batchable matrix product — which requires one explicit transition matrix per symbol, so its cost driver is $2^{|\mathcal{P}|}$. Neither weakness can be swapped onto the other paradigm without discarding the very representational choice that defines it; the factored representation (Section 4.4) is precisely the attempt to compress *both* axes at once, and recovers the flat-in-$|\mathcal{P}|$ scaling on the read-once guards where such a compression exists.

TODO: capability matrix summarizing exactness, soft-input support, differentiability, and the three heels across the three paradigms.

5.2 Repaired RuleRunner and DeepDFA as Learnable Objects

As *monitors*, the completeness-restored RuleRunner and DeepDFA are interchangeable: the progression construction converges to the same minimal DFA (Section 3.3), so on any crisp trace they agree, and on crisp throughput the symbolic walk is optimal for both. The reason to keep both lineages is not runtime but what they expose to gradient-based adaptation.

DeepDFA's learnable parameters are the entries of the transition tensor and the accepting vector: a gradient adapts transition probabilities between *opaque* states, over $|Q|^2 \cdot |\mathcal{P}|$ parameters with no structural inductive bias, and the learned object can drift into an arbitrary weighted automaton no longer corresponding to any LTL_f formula. The rule-based construction, by contrast, attaches parameters to the *rules*, indexed by subformula and operator at the symbolic construction level: adaptation is syntactically local, the learned object re-extracts as a corrected specification, and the parameterization carries an inductive bias toward valid specifications. The two are thus interchangeable as monitors but not as learnable objects — DeepDFA adapts an opaque automaton, the rule form adapts a structured specification — which is the distinction we build on in the specification-adaptation experiments (Section 6) and the bridge to follow-up work.

TODO: expand once the adaptation experiments are in; cite Bacchus–Kabanza progression and the LTLf 2EXP bound as primary sources.

6 Experimental Comparison

All three monitoring paradigms are implemented in a common framework sharing a uniform MONITOR interface, which ensures that the experimental harness is paradigm-agnostic: timing experiments iterate over a list of monitor instances without any paradigm-specific branching. The shared compilation pipeline uses `ltlf2dfa` [3] to translate each LTL_f formula into a minimal DFA once at construction time.

We evaluate the monitoring paradigms along three axes, using synthetic traces throughout Experiments 1–3 to isolate per-step monitoring cost from data-dependent effects such as early-termination frequency. This follows the methodology of experiments in [11].

7 Related Work

8 Conclusion and Future Work

In this paper ...

8.1 Transformers

8.2 Specification Adaptation

8.3 Process Model Repair

References

[1] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Trans. Softw. Eng. Methodol.*, 20(4):14:1–14:64, 2011.

[2] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In Francesca Rossi, editor, *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, pages 854–860. IJCAI/AAAI, 2013.

[3] Francesco Fuggitti. Ltlf2dfa. *Zenodo*, 2019.

[4] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.

[5] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. The glory of the past. In *Workshop on Logic of Programs*, pages 196–218. Springer, 1985.

[6] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017.

[7] Nikolaos Manginas, George Paliouras, and Luc De Raedt. Nesya: Neurosymbolic automata. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2025, Montreal, Canada, August 16-22, 2025*, pages 5950–5958. ijcai.org, 2025.

[8] Zohar Manna and Amir Pnueli. *Temporal Verification of Reactive Systems*, volume 1. Springer-Verlag, 1995.

[9] Axel Mezini, Elena Umili, Ivan Donadello, Fabrizio Maria Maggi, Matteo Mancanelli, and Fabio Patrizi. Neuro-symbolic predictive process monitoring. *arXiv preprint arXiv:2509.00834*, 2025.

[10] Alan Perotti, Guido Boella, and Artur d'Avila Garcez. Runtime verification through forward chaining. In *Runtime Verification: 6th International Conference, RV 2015, Vienna, Austria, September 22-25, 2015. Proceedings*, pages 185–200. Springer, 2015.

[11] Alan Perotti, Artur d'Avila Garcez, and Guido Boella. Neural networks for runtime verification. In *2014 International Joint Conference on Neural Networks (IJCNN)*, pages 2637–2644. IEEE, 2014.

[12] Alan Perotti, Artur d'Avila Garcez, and Guido Boella. Neural-symbolic monitoring and adaptation. In *2015 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2015.

[13] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*, pages 46–57. IEEE Computer Society, 1977.

[14] Elena Umili and Roberto Capobianco. Deepdfa: Automata learning through neural probabilistic relaxations. In *ECAI 2024 - 27th European Conference on Artificial Intelligence, 19-24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*, pages 1051–1058, 2024.